mSEC-AT

LLM Prompt Catalogue

Structured overview of the prompts used for requirement assessment, report generation, translation, and justification drafting

Document purpose	Reference catalogue for the LLM prompts used in the mSEC-AT audit workflow
Scope	Prompt role, input parameters, output constraints, and complete prompt templates
Prepared date	24 Jun 2026

Contents

Overview of the LLM prompts used in mSEC-AT

mSEC-AT uses four LLM prompts at different stages of the audit workflow. Each prompt has a specific role and is constrained to operate on structured inputs generated by the pipeline. The prompts are not used as unconstrained text generators; instead, they support requirement-level assessment, report generation, translation, and justification drafting while preserving traceability to the evidence collected by the tool.

The first prompt is used to translate non-English requirement descriptions into English. It receives requirement identifiers and original descriptions as compact JSON-as-text input and returns English translations in JSON format. This ensures that the generated audit spreadsheet remains consistent with the English-only output requirement.

The second prompt is used for requirement-level security assessment. It instructs the model to act as a senior mobile security auditor and to evaluate one security requirement at a time using requirement metadata and evidence collected from the application analysis. Its output is a strict JSON object containing a decision, evidence, rationale, and, when needed, manual verification steps.

The third prompt is used to draft English justifications for requirement-level outcomes in the generated Excel file. The requirement result is computed before this prompt is invoked; therefore, the model is not responsible for deciding whether a requirement is compliant, non-compliant, or not applicable. Its role is explanatory: it produces concise justifications based only on the precomputed result, mapped fingerprint flags, observed summaries, notes, evidence counts, and decision metadata.

The fourth prompt is used during audit summary generation. It does not classify requirements. Instead, it receives weakness patterns, prevalence counts, severity, likelihood, recommended ownership, and evidence anchors derived from the audit workbook. Its purpose is to improve the wording of the report and generate actionable recommendations without inventing controls, metrics, or facts beyond the provided data.

Together, these prompts support different parts of the mSEC-AT pipeline while maintaining a conservative use of LLMs. Decisions and metrics are grounded in pipeline-derived evidence, and the LLM is mainly used to structure, translate, or explain outputs in a controlled and machine-readable format.

This catalogue is intended to document the LLM prompts and their controlled use within mSEC-AT. Prompt outputs are constrained to JSON so that they can be processed automatically by the pipeline while preserving traceability to pipeline-derived evidence.

Prompt inventory

#	Prompt	Source file	Role in the pipeline	Output
1	**English translation**	ai_security_audit_requirements_excel.py	Translates non-English requirement descriptions into English.	Strict JSON: translated text_en items
2	**Requirement-level security assessment**	ai_correlate.py	Classifies one requirement using metadata and analysis evidence.	Strict JSON decision: YES - NO - N/A - MANUAL
3	**Excel justification drafting**	ai_security_audit_requirements_excel.py	Drafts justifications for precomputed requirement outcomes.	Strict JSON: justification per requirement
4	**Audit summary prose and recommendations**	audit_summary_stage2_generate_docx.py	Improves report wording and generates recommendations from weakness patterns.	Strict JSON: key_takeaways and pattern_writeups

Model configuration and backend independence

The model configuration defines the execution profiles and the LLM parameters used by the pipeline, including the model provider, model identifier, API endpoint, API key environment variable, reasoning effort level, maximum output token limit, and temperature settings. The LLM call is configured to provide the model with the required input data, define its expected behaviour during the task, and enforce structured and validated outputs rather than free-form text.

Depending on the prompt, the system message defines the role and constraints of the model, such as acting as a senior mobile security auditor, audit reporting specialist, translator, or justification drafter. Across all cases, the prompts constrain the model to use only the provided context, avoid inventing findings or metrics, and return the output in the required structured format. Depending on the prompt, the user message provides the required task-specific data as a JSON payload. For requirement-level assessment, this includes the requirement identifier, requirement metadata, available evidence, mapped flags, notes, and decision metadata. For justification drafting, it also includes the precomputed result generated by the deterministic evaluation logic. The expected response format is specified through a schema, allowing the SDK to parse and validate the generated output automatically.

This design allows mSEC-AT to operate with different LLM backends, including cloud-based APIs and locally deployed models, such as Ollama or vLLM, while maintaining a consistent interface.

Prompt-level invocation configuration

The following subsections document the call-level configuration used for each prompt, including the invocation method, input format, model source, endpoint, reasoning effort, output-token limit, temperature setting, and expected structured output. Across the pipeline, the model identifier is read from LLM_MODEL, and the OpenAI-compatible client is configured through environment variables such as LLM_API_KEY and, where applicable, LLM_BASE_URL. In the evaluated configuration, temperature was set to 0 to reduce stochastic variation. Output variability is also controlled through constrained prompts, structured JSON outputs, reasoning-effort settings, maximum output-token limits, and deterministic downstream validation.

1. English translation prompt

Source file: scripts/ai_security_audit_requirements_excel.py

LLM invocation configuration

Parameter	Value / source
Invocation method	client.responses.parse, with client.responses.create fallback
Source file	scripts/ai_security_audit_requirements_excel.py
Input format	System message + JSON user payload
Model	Environment variable: LLM_MODEL
API key	Environment variable: LLM_API_KEY
API endpoint	Environment variable: LLM_BASE_URL
Reasoning effort	Environment variable: LLM_REASONING_EFFORT
Default reasoning effort	medium
Maximum output tokens	Environment variable: LLM_MAX_OUTPUT_TOKENS
Default token limit	2000
Temperature	0
Output format	Strict JSON
Expected output fields	items[id, text_en]

This configuration is intended to reduce output variability while ensuring that translations preserve the original requirement meaning and are returned as structured JSON.

This prompt translates short requirement descriptions into English when non-English requirement descriptions are detected and English-only output is required. It receives requirement identifiers and original requirement text as compact JSON-as-text input.

The model must preserve the original meaning, avoid adding new requirements, and return only JSON. The translated descriptions are then used by the pipeline to keep generated audit artefacts consistent with the English-only output requirement.

Main role

Ensures that requirement descriptions used in generated artefacts are available in concise professional English.

Input parameters

Parameter	Source	Purpose / meaning
{items}	Translation batch	JSON array containing requirement identifiers and source text.
{puid}	Requirement object	Identifier of the requirement whose description needs translation.
{requirement_description}	Requirement description field	Original requirement description extracted from the requirements file.

System message

```
You translate short requirement descriptions to English.

Strict rules:
- Output English only.
- Preserve meaning. Do not add new requirements.
- Keep the translation concise and professional.
- Return ONLY JSON in the form: {"items": [{"id": "...", "text_en": "..."}, ...]}.
```

User payload

```
{
  "items": [
    {
      "id": "{puid}",
      "text": "{requirement_description}"
    }
  ]
}
```

2. Requirement-level security assessment prompt

Source file: scripts/ai_correlate.py

LLM invocation configuration

Parameter	Value / source
Invocation method	client.responses.create
Source file	scripts/ai_correlate.py
Input format	Single prompt string generated for one requirement at a time
Model	Environment variable: LLM_MODEL
API key	Environment variable: LLM_API_KEY
API endpoint	Environment variable: LLM_BASE_URL
Reasoning effort	Environment variable: LLM_REASONING_EFFORT
Default reasoning effort	medium
Maximum output tokens	Environment variable: LLM_MAX_OUTPUT_TOKENS
Default token limit	900
Temperature	0
Output format	Strict JSON
Expected output fields	decision, evidence, rationale, manual_steps

This configuration is intended to reduce output variability while keeping the requirement-level decision constrained to a structured and automatically parseable JSON object.

This prompt supports requirement-level security assessment. It instructs the model to act as a senior mobile security auditor specialised in Android applications and to evaluate one security requirement at a time. The output is constrained to strict JSON so that it can be automatically processed by the pipeline and integrated into the generated audit artefacts.

The prompt receives both requirement metadata and technical evidence collected during the analysis. It follows a conservative decision policy: YES is expected only when the requirement applies and supporting evidence exists, NO is expected when collected evidence contradicts the requirement, N/A is used when the required capability is absent, and MANUAL is used when the evidence is insufficient or runtime verification is required.

Main role

Transforms heterogeneous technical evidence into a structured requirement-level decision using a conservative decision policy.

Input parameters

Parameter	Source	Purpose / meaning
{puid}	Current requirement object	Identifier of the requirement being evaluated.
{desc}	Requirement description field	Textual description of the security requirement.
{crit}	Validation criteria field	Criteria used to evaluate the requirement.
{importance}	Requirement importance field	Importance value, or Not described when unavailable.
{app_ctx}	Application evidence summary	Compact summary of SARIF, MobSF, Trivy, Manifest, exported components, and network security configuration.
{code_hints_set}	Manifest and code-hint extraction	Exported-component details and detected risk patterns.
{code_hints_top}	Source-code hint extraction	Examples of relevant source-code matches.
{sarif_top}	SARIF evidence	Most relevant SARIF rule counts.
{mobsf_set}	MobSF evidence	Security hints extracted from MobSF reports.
{trivy_brief}	Trivy evidence	Compact JSON summary of Trivy findings.

Evidence flags and deterministic adjudication policy

mSEC-AT derives compliance decisions from the set of evidence flags associated with each requirement rather than from a single isolated indicator. These flags encode positive controls, negative risk signals, and applicability conditions. Rather than being assigned in an ad hoc manner, they were iteratively refined through expert review and pilot testing before being incorporated into mSEC-AT. They are represented through explicit and traceable rules that identify the source of the underlying evidence.

The evaluation mechanism follows several principles. Lack of coverage by a scanner is not automatically interpreted as a negative result. The relationships between requirements and flags are kept stable to preserve comparability across audits. Evidence from new tools is integrated into existing flags when its meaning is equivalent, while new signals can be exposed as optional flags for future extensions of the catalogue. This separation between the requirement-flag mapping and the rules used to compute flag values facilitates the adaptation of the approach to other audit contexts, as it allows new regulatory requirements, analysis tools, or domain-specific signals to be incorporated without fully redesigning the evaluation model.

mSEC-AT applies a deterministic adjudication policy over the evidence flags associated with each requirement. A requirement is classified as compliant only when the applicable evidence supports the expected secure posture and no conflicting negative risk signals are present. If at least one negative risk signal contradicts the expected secure posture, a conservative precedence rule is applied, and the requirement is classified as non-compliant. When the available evidence is absent, unknown, ambiguous, or only partially confirms the requirement, the requirement is not automatically assigned a positive compliance label and is instead marked for manual review when expert assessment is needed.

Duplicate findings are retained to preserve end-to-end traceability with respect to the underlying security analysis tools. However, they are consolidated at the requirement level and do not independently influence the final compliance decision.

For example, consider the requirement “Data shared between devices must be encrypted.” If the available evidence confirms encrypted HTTP communications but provides no evidence regarding other communication channels, the requirement cannot be automatically classified as compliant. Instead, it is marked for manual review so that the auditor can determine whether the requirement is fully satisfied across all relevant communication mechanisms.

Complete prompt template

```
You are a Senior Mobile Security Auditor specialized in Android apps. Evaluate ONE
requirement.

OUTPUT (STRICT JSON):
- "decision": "YES"|"NO"|"N/A"|"MANUAL"
- "evidence": 1-5 bullets (short)
- "rationale": 2-5 bullets (short, no chain-of-thought)
- "manual_steps": only if "MANUAL"

RULES:
- "N/A" if the capability clearly doesn't exist in app context.
- "NO" if there is contradictory evidence vs the requirement (e.g., allowBackup=true;
debuggable=true; exported components w/ IF; cleartext allowed).
- "YES" only if requirement applies and there's positive evidence or absence of
contradictions (be conservative).
- "MANUAL" when runtime behavior or evidence is insufficient/ambiguous.

FEW-SHOTS omitted for brevity (same as previous message).

CURRENT CASE
PUID: {puid}
Requirement description: {desc}
Validation criteria: {crit}
Importance: {importance}
App context: {app_ctx}

Evidence:
- Manifest/Exported: {code_hints_set}
- Code hints (examples): {code_hints_top}
- SARIF: {sarif_top}
- MobSF: {mobsf_set}
- Trivy: {trivy_brief}

Return STRICT JSON only.
```

3. Excel justification drafting prompt

Source file: scripts/ai_security_audit_requirements_excel.py

LLM invocation configuration

Parameter	Value / source
Invocation method	client.responses.parse, with client.responses.create fallback
Source file	scripts/ai_security_audit_requirements_excel.py
Input format	System message + JSON user payload
Model	Environment variable: LLM_MODEL
API key	Environment variable: LLM_API_KEY
API endpoint	Environment variable: LLM_BASE_URL
Reasoning effort	Environment variable: LLM_REASONING_EFFORT
Default reasoning effort	medium
Maximum output tokens	Environment variable: LLM_MAX_OUTPUT_TOKENS
Default token limit	2000
Temperature	0
Output format	Strict JSON
Expected output fields	items[id, justification]

This configuration is intended to reduce output variability while ensuring that justifications explain the precomputed requirement outcome without changing the underlying decision.

This prompt drafts English audit justifications for requirement-level outcomes in the generated Excel file. The final requirement result is computed before the prompt is invoked; therefore, the model is not responsible for deciding whether a requirement is compliant, non-compliant, or not applicable.

The model receives the precomputed result, mapped fingerprint flags, observed summaries, notes, evidence counts, and decision metadata. Its role is explanatory rather than decisional: it improves readability while preserving the result already computed by the pipeline.

Main role

Converts deterministic audit results and fingerprint-derived evidence into concise English justifications for the Excel output.

Input parameters

Parameter	Source	Purpose / meaning
{puid}	Requirement audit context	Identifier of the requirement being justified.
{description_en}	Translated or original English description	English requirement description used in the Excel output.
{result}	Precomputed pipeline result	Requirement result, such as yes, no, or n/a.
{flags_used}	Requirement-to-flag mapping	List of fingerprint flags mapped to the requirement.
{meta}	Decision metadata	Additional metadata such as prohibitive, conditional, or feature-gating logic.
{flag_id}	Fingerprint flag	Identifier of the flag used as evidence.
{classification}	Flag interpretation logic	Indicates whether the flag is a positive control, negative risk, or applicability signal.
{expected}	Flag expectation logic	Expected normalized value, usually YES for positive controls and NO for negative risks.
{observed_summary_norm}	Fingerprint verdict parsing	Normalized observed value extracted from the fingerprint: YES, NO, or NA.
{state}	app_verdict field	Original verdict state associated with the flag.
{summary}	app_verdict field	Original app_verdict summary.
{notes}	app_verdict field	Notes associated with the flag, paraphrased when necessary.
{evidence_count}	app_verdict field	Number of evidence items associated with the flag.
{note_hint}	Derived hint	Indicates when Fallback verdict is present.
{outcome}	Evaluation against expected value	Indicates whether the observed flag supports, contradicts, is missing, or is unknown.

System message

```
You draft audit justifications in English for security requirement outcomes.

Strict rules:
- Output English only.
- If any provided notes contain non-English text, paraphrase them into English and do not
quote them verbatim.
- Do not invent evidence or flags.
- Use only the provided context.
- Keep 1 to 3 sentences per requirement.
- Explicitly mention: app_verdict.state, normalized app_verdict.summary (YES/NO/NA),
notes (include 'Fallback verdict' if present), and evidence_count.
- If a flag is not present in the fingerprint, state: 'flag not present in fingerprint'.
- Do not change the precomputed result.
- Return ONLY JSON in the form: {"items": [{"id": "...", "justification": "..."}, ...]}.
```

User payload

```
{
  "batch": [
    {
      "id": "{puid}",
      "description_en": "{description_en}",
      "result": "{result}",
      "flags_used": "{flags_used}",
      "meta": "{meta}",
      "flags": [
        {
          "id": "{flag_id}",
          "classification": "{classification}",
          "expected": "{expected}",
          "observed_summary_norm": "{observed_summary_norm}",
          "app_verdict": {
            "state": "{state}",
            "summary": "{summary}",
            "notes": "{notes}",
            "evidence_count": "{evidence_count}",
            "note_hint": "{note_hint}"
          },
          "outcome": "{outcome}"
        }
      ]
    }
  ]
}
```

4. Audit summary prose and recommendation prompt

Source file: scripts/audit_summary_stage2_generate_docx.py

LLM invocation configuration

Parameter	Value / source
Invocation method	client.responses.create
Source file	scripts/audit_summary_stage2_generate_docx.py
Input format	System message + JSON user payload
Model	Environment variable: LLM_MODEL
API key	Environment variable: LLM_API_KEY
API endpoint	Environment variable: LLM_BASE_URL
Reasoning effort	Environment variable: LLM_REASONING_EFFORT
Default reasoning effort	medium
Maximum output tokens	Environment variable: LLM_MAX_OUTPUT_TOKENS
Default token limit	6000
Temperature	0
Output format	Strict JSON
Expected output fields	key_takeaways, pattern_writeups

This configuration is intended to reduce output variability while keeping the generated report prose grounded in workbook-derived patterns, counts, evidence anchors, and the required JSON schema.

This prompt is used during generation of the audit summary report. Unlike the requirement-level prompt, it is not used to classify individual requirements. Instead, it is a style-enhancement and reporting-support step that generates paper-quality prose components and actionable recommendations from weakness patterns already derived from the audit workbook.

The model receives weakness-pattern data, including prevalence counts, severity, likelihood, recommended owner, and evidence anchors. It is explicitly instructed not to invent implemented controls, not to invent metrics, and not to claim facts beyond the provided anchors and counts.

Main role

Turns workbook-derived weakness patterns into structured prose elements for key takeaways, expected-control descriptions, impact statements, and recommendations.

Input parameters

Parameter	Source	Purpose / meaning
{max_takeaways}	Function argument	Maximum number of key findings requested from the model.
{likelihood_rubric}	Audit analysis package	Defines how likelihood labels are assigned.
{pattern}	Weakness-pattern record	Name of the weakness pattern.
{mapped_noncompliant_count}	Weakness-pattern record	Number of non-compliant controls mapped to the pattern.
{severity}	Weakness-pattern record	Severity assigned to the pattern.
{likelihood}	Derived from prevalence count	Likelihood level computed from mapped non-compliance count.
{recommended_owner}	Weakness-pattern record	Team or role expected to address the weakness.
{description_anchors}	Workbook-derived evidence	Short evidence anchors that constrain the generated prose.

System message

```
You are a senior security audit reporting specialist. You will improve wording and generate actionable recommendations ONLY at the weakness-pattern level. You must NOT invent specific implemented controls. You must NOT invent metrics. You must NOT claim facts beyond the provided anchors and counts. Return strict JSON only.
```

User payload

```
{
  "task": "Generate paper-quality prose components grounded in workbook-derived
prevalence counts.",
  "constraints": {
    "no_invented_metrics": true,
    "no_category_level_bullet_dumps": true,
    "no_long_id_lists": true,
    "recommendations_no_time_headings": true,
    "max_key_takeaways": {max_takeaways},
    "likelihood_rubric": {likelihood_rubric}
  },
  "input_patterns": [
    {
      "pattern": "{pattern}",
      "count": "{mapped_noncompliant_count}",
      "severity": "{severity}",
      "likelihood": "{likelihood}",
      "owner": "{recommended_owner}",
      "anchors": "{description_anchors}"
    }
  ],
  "required_output_schema": {
    "key_takeaways": [
      "<5-7 bullets; each references prevalence count and pattern name>"
    ],
    "pattern_writeups": [
      {
        "pattern": "<exact pattern name from input>",
        "expected": "<1-2 sentences>",
        "impact": "<1-2 sentences; CIA + privacy/regulatory context where relevant>",
        "recommendations": [
          "<6-10 bullets; practical; may include MFA/biometric step-up if appropriate>"
        ]
      }
    ]
  }
}
```